

Blatt 3

Web-Security und Intrusion Prevention

Luis Staudt

Aufgabe 1: Die Webanwendung

Die Flask-Anwendung `app.py` listet ohne GET-Parameter alle Dateien im Unterordner `files/` auf und verlinkt sie über den Parameter `file`. Ist `file` gesetzt, gibt die Anwendung die angegebene Datei direkt mit `send_file` zurück:

```
1 @app.route('/')
2 def index():
3     if not request.args.get("file"):
4         return generate_list()
5     else:
6         return send_file(request.args.get("file"))
```

Da der übergebene Pfad ungeprüft an `send_file` gelangt, lassen sich beliebige Pfade einschleusen.

Aufgabe 2: Die Anwendung ausnutzen

Der Parameter darf ein absoluter Pfad sein:

```
1 curl "http://127.0.0.1:5000/?file=/etc/passwd"
2 -> root:x:0:0:root:/root:/bin/bash
3     daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
4     ...
```

Ebenso lässt sich mit ../ aus dem Arbeitsverzeichnis heraus navigieren (Path Traversal):

```
1 curl "http://127.0.0.1:5000/?file=files ../../../../etc/passwd"
2 -> root:x:0:0:root:/root:/bin/bash
3     ...
```

Beide Aufrufe geben den Inhalt von /etc/passwd preis. Analog lassen sich andere Dateien auslesen, etwa app.py selbst.

Aufgabe 3: Absicherung

Erster Versuch: Präfix-Prüfung

Geprüft wird, ob der Pfad mit `files/` beginnt:

```
1 if not file.startswith("files/"):
2     abort(403)
3 return send_file(file)
```

Diese Prüfung blockiert absolute Pfade (`/etc/passwd` → 403), ist aber unzureichend: Ein relativer Pfad beginnt ebenfalls mit `files/` und umgeht die Prüfung trotzdem.

```
1 curl -o /dev/null -w "%{http_code}" "http://.../?file=/etc/passwd"
2 -> 403 (blockiert)
3 curl "http://.../?file=files/../../../../etc/passwd"
4 -> 200 (Datei wird weiterhin ausgegeben)
```

Die Ursache sind relative Pfade: Das Präfix `files/` wird durch `../` sofort wieder verlassen.

Sichere Lösung mit `os.path.realpath()`

Der übergebene Pfad wird vollständig aufgelöst, wobei `..` und Symlinks entfernt werden, und muss tatsächlich innerhalb des erlaubten Verzeichnisses liegen:

```
1 import os
2 BASE_DIR = os.path.realpath("files")
3
4 @app.route('/')
5 def index():
6     file = request.args.get("file")
7     if not file:
8         return "<h1>Keine Datei angegeben</h1>"
9     requested = os.path.realpath(file)
10    if requested != BASE_DIR and not requested.startswith(BASE_DIR + os.sep):
11        abort(403)
12    return send_file(requested)
```

Da `realpath` den Pfad kanonisiert, bevor er mit dem Basisverzeichnis verglichen wird, scheitern sowohl absolute als auch relative Traversal-Pfade. Weil Symlinks mit aufgelöst werden, schützt die Prüfung zugleich gegen Angriffe über symbolische Links.

```
1 curl -o /dev/null -w "%{http_code}" "http://.../?file=/etc/passwd"
2 -> 403 (blockiert)
3 curl -o /dev/null -w "%{http_code}" "http://.../?file=files/../../../../etc/passwd"
4 -> 403 (blockiert)
5 curl "http://.../?file=files/beispiel.txt"
6 -> Dies ist eine harmlose Beispieldatei. (erlaubte Datei weiterhin verfuegbar)
```

Zusammenfassung der drei Stufen

	absoluter Pfad	relativer Traversal
app.py (verwundbar)	200 (Leak)	200 (Leak)
app_naiv.py (Präfix)	403	200 (Bypass)
app_secure.py (realpath)	403	403